

Testing Guide

Revision: 4 **Last modified:** 2026-08-19T17:00:43Z

This is the **authoritative test-type catalogue** for qBittorrent-Fixed. Every testable module in the repo must have coverage in every applicable row below. The catalogue is derived from Part C of the completion-initiative plan (docs/superpowers/plans/2026-04-19-completion-initiative.md).

Bootstrap (one-time virtualenv setup)

Test dependencies are **not** installed system-wide. Create and populate a local .venv/ (gitignored – see .gitignore) before running anything below:

```
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r tests/requirements.txt
```

Verified working combination (2026-08-08): Python 3.14.6 (/usr/bin/python3.14) + pytest 9.1.1 + pytest-asyncio 1.4.0 + fastapi 0.141.1 + pydantic 2.13.4 + every package listed in tests/requirements.txt, all installed fresh into the same virtualenv (no cross-ABI mixing). Re-run the pip install line whenever tests/requirements.txt changes – it is idempotent.

Once bootstrapped, invoke pytest via .venv/bin/python -m pytest ... (or activate the venv first with source .venv/bin/activate and drop the .venv/bin/ prefix from every command in this guide).

Test types

#	Type	Framework	Directory	How to run	Coverage / report land
1	Unit (Python)	pytest + pytest-asyncio	tests/unit/	pytest tests/unit/ -v --import-mode=importlib	htmlcov/, coverage.xr
2	Unit (Frontend)	Vitest + Angular Testing Library	frontend/src/**/*.spec.ts	cd frontend && ng test	frontend/coverage/
3	Integration (API)	pytest + httpx.AsyncClient	tests/integration/	pytest tests/integration/ -v --import-mode=importlib	htmlcov/, coverage.xr
4	Integration (Plugin – i.e. nova3)	pytest + vcrpy (record/replay)	tests/integration/plugins/	pytest tests/integration/plugins/ -v	htmlcov/

#	Type	Framework	Directory	How to run	Coverage / report land
			(pending Phase 4)		
5	End-to-end (browser)	Playwright (Python)	tests/e2e/	pytest tests/e2e/ -v	artifacts/e2e/
6	End-to-end (API flow)	pytest + live compose	tests/e2e/ (fixture-gated)	pytest tests/e2e/ -v --import-mode=importlib	htmlcov/
7	Contract (OpenAPI)	schemathesis	tests/contract/	schemathesis run http://localhost:7187/openapi.json	artifacts/contract/
8	Property-based hypothesis		tests/property/ (pending)	pytest tests/property/ -v	htmlcov/
9	Fuzz	hypothesis + atheris (optional)	tests/fuzz/ (pending)	pytest tests/fuzz/ -v	artifacts/fuzz/
10	Mutation	mutmut	runs against download-proxy/src/ (pending)	mutmut run	html/mutmut.html
11	Security	pytest + bandit + semgrep rules	tests/security/	pytest tests/security/ -v	htmlcov/ + artifacts/scans/
12	Performance (latency)	pytest-benchmark	tests/performance/, tests/benchmark/	pytest tests/performance/ tests/benchmark/ -v --benchmark-only	.benchmarks/
13	Load (throughput)	Locust	tests/load/ (pending Phase 6)	locust -f tests/load/locustfile.py	artifacts/load/
14	Stress (overload)	Locust + chaos probes	tests/stress/	pytest tests/stress/ -v	artifacts/stress/
15	Chaos (fault injection)	toxiproxy + pytest	tests/chaos/ (pending)	pytest tests/chaos/ -v	artifacts/chaos/
16	Concurrency (race)	pytest-randomly + pytest-repeat + asyncio ops	tests/concurrency/ (pending)	pytest tests/concurrency/ -v --count=10	htmlcov/
17	Memory leak	tracemalloc + pytest + objgraph	tests/memory/ (pending)	pytest tests/memory/ -v	artifacts/memory/
18	Deadlock / timeout	pytest-timeout (hard cap all tests)	global	pytest -v (30 s timeout is the default in pyproject.toml)	test log
19	Smoke	pytest			htmlcov/

#	Type	Framework	Directory	How to run	Coverage / report land
			tests/smoke/ (pending)	pytest tests/smoke/ -v	
20	Monitoring / metrics	pytest + Prometheus scrape asserts	tests/observability/ (pending)	pytest tests/observability/ -v	htmlcov/
21	Infra / compose	pytest-docker + compose-config validator	tests/infra/ (pending)	pytest tests/infra/ -v	htmlcov/
22	Accessibility	Playwright + axe-core	tests/ally/ (pending)	pytest tests/ally/ -v	artifacts/ally/
23	Visual regression	Playwright snapshots	tests/visual/ (pending)	pytest tests/visual/ --update-snapshots	tests/visual/ __snapshots/
24	Documentation link check	pytest + linkchecker	tests/docs/ (pending)	pytest tests/docs/ -v	artifacts/docs/
25	Type-check (static)	mypy --strict + tsc --noEmit	CI job	mypy download-proxy/src plugins / cd frontend && tsc --noEmit	CI log
26	Lint (static)	ruff + eslint + shellcheck + hadolint + yamllint	CI job	ruff check . / cd frontend && ng lint / shellcheck *.sh	CI log
27	Dependency audit	pip-audit + npm audit + Snyk + Trivy	CI job	pip-audit -r download-proxy/requirements.txt / snyk test	artifacts/scans/
28	SAST	Semgrep + Bandit + SonarQube	CI job	semgrep scan --config=auto / bandit -r download-proxy/src	artifacts/scans/
29	Secret scan	Gitleaks + TruffleHog	CI job	gitleaks detect	artifacts/scans/
30	License compliance	pip-licenses + license-checker	CI job	pip-licenses / cd frontend && license-checker	artifacts/licenses/

Rows marked *pending* are scaffolded by the completion-initiative plan phases (see the plan document for the phase that ships each one).

Where tests live

- **Top-level tests/** -- all merge-service tests, *not* download-proxy/tests/ (which does not exist).

- **frontend/src/**/*.spec.ts** â€” co-located Angular unit tests.
- **CI jobs** â€” .github/workflows/{syntax,unit,integration,nightly,security}.yml.

Service-availability fixtures

Since Phase 0 the following fixtures **start the compose stack** rather than skip when services are down (converted 71 runtime skips to fixture-gated executions, per commit 55d29ce):

Fixture	Starts	Asserts
merge_service_live	qbittorrent-proxy	GET http://localhost:7187/health â†’ 200
qbittorrent_live	qbittorrent	GET http://localhost:7186/ â†’ 200
webui_bridge_live	webui-bridge.py	GET http://localhost:7188/health â†’ 200
all_services_live	all three	all of the above

Defined in tests/fixtures/services.py and wired into tests/conftest.py.

How to add a new test (TDD cadence)

The CLAUDE.md critical constraint is non-negotiable:

1. **RED** â€” write the failing test first. Name it after the behaviour, not the implementation. Put it in the directory that matches the test type from the catalogue above.
2. **Watch it fail** â€” run the test in isolation (pytest tests/unit/test_new_thing.py::test_specific_case -v) and confirm the failure reason is the one you want (NOT an import error, NOT a fixture error).
3. **GREEN** â€” write the minimum production code to make the test pass. No refactors yet.
4. **Verify** â€” run the whole affected test directory to prove nothing else regressed.
5. **Rebuild-reboot** â€” if the change touched container code, follow the CLAUDE.md REBUILD AND REBOOT constraint:

```
./stop.sh
podman exec qbittorrent-proxy find / -name __pycache__ -type d -exec rm -rf {} \;
./start.sh -p
```

Curl-check the served content matches the committed code.

6. **Commit** â€” one behaviour per commit. Commit message follows `<type>(phase-N): <subject>` when part of a phased plan.

Coverage gate

- `pyproject.toml` contains `[tool.coverage.report]` with `fail_under` set to the current gate (Phase 0 starts low; Phase 5 raises it module-by-module to 100 %).
- The baseline zero point is captured in [COVERAGE_BASELINE.md](#).
- HTML reports land in `htmlcov/`; XML (for SonarQube) in `coverage.xml`.

CI entry points

- **Local full pipeline** â€” `./ci.sh`
- **Quick loop** â€” `./ci.sh --quick`
- **Hardcoded-podman runner** â€” `./run-all-tests.sh` (note: hardcodes podman, will fail on docker-only systems â€” see [CLAUDE.md](#) gotcha)
- **Single-file** â€” `./test.sh --plugin <name>` or `./test.sh --full`

Per-tracker search stats

Tests that cover `TrackerSearchStat` and the `tracker_started / tracker_completed` SSE events live across four suites:

Test file	Type	What it pins
<code>tests/unit/merge_service/test_tracker_stats.py</code>	Unit	<code>TrackerSearchStat</code> defaults, status transitions (pending â†’ running â†’ success / empty / error / timeout), <code>duration_ms</code> bookkeeping, <code>auth_flag</code> provenance, and sorted <code>to_dict()</code> serialisation. The SSE poll loop emits <code>tracker_started</code> exactly once on
<code>tests/unit/api_layer/test_tracker_stats_sse.py</code>	Unit (streaming)	<code>pendingâ†’running</code> and <code>tracker_completed</code> exactly once per terminal flip; gracefully degrades when metadata predates <code>tracker_stats</code> .
<code>tests/property/test_tracker_stats_properties.py</code>	Property (Hypothesis)	<code>sum of results_count == total_results</code> , every completed stat has

Test file	Type	What it pins
tests/contract/ test_tracker_stats_contract.py	Contract	duration_ms >= 0, set(tracker_stats keys) == set(trackers_searched), failed trackers retain error_type + error. POST /api/v1/search, POST /api/v1/search/ sync, and GET /api/v1/ search/{id} all expose the 15-field tracker_stats payload on SearchResponse.

The frontend dialog + chip bar are covered by frontend/src/app/components/tracker-stat-dialog/tracker-stat-dialog.component.spec.ts and the tracker stats bar describe block in frontend/src/app/components/dashboard/dashboard.component.spec.ts.

Runtime theme (palette system)

The runtime-switchable palette system (see [docs/DESIGN_SYSTEM.md](#)) is guarded by a four-tier test bundle “every tier asserts *observable* state (DOM, CSS variables, localStorage, Playwright computed style) per the “no false positives” mandate:

Test file	Type	What it pins
tests/unit/ test_palette_catalog.py	Unit (parametric)	Every palette has both variants, every one of the 15 tokens is a valid CSS colour, ids are unique, DEFAULT_PALETTE_ID resolves, Darcula accent is #9d001e. Per-palette parametrisation names the offender. :root fallback block covers every token; dashboard SCSS uses var(--color-*) instead of \$accent/\$bg-* Sass variables; dashboard template includes <app- theme-picker>; TOKEN_CSS_VAR map covers every var.
tests/unit/ test_theme_wiring.py	Unit (integration)	System-dark detection, stored-state hydration,
	Unit (Vitest)	

Test file	Type	What it pins
frontend/src/app/ services/ theme.service.spec.ts		unknown-id fallback, setPalette writes all 15 tokens with exact values to documentElement.style, toggleMode flips signal + data-mode + color- scheme, prefers-color- scheme change listener respects modeIsUserChosen flag.
frontend/src/app/ components/theme-picker/ theme- picker.component.spec.ts	Unit (Vitest)	One menu item per palette, swatches render with accent/contrast/bg/ text colours, clicking an item calls setPalette, clicking outside closes the menu, active palette has .active + aria- checked="true". Live dashboard at :7187: picking a new palette changes --color-accent (+3 others) on <html> to the catalogue value AND writes qbit.theme into localStorage. Skips with a loud message when the served bundle predates the theme-picker.
tests/e2e/ test_theme_runtime.py	End-to-end (Playwright)	

Whenever the TrackerSearchStat shape changes, run ./scripts/freeze-openapi.sh to refresh docs/api/openapi.json so the contract test test_frozen_and_live_have_same_schemas stays green.

Gotchas

- pytest needs --import-mode=importlib for the merge service "the project is not a Python package (no __init__.py at the root).
- Integration tests with requests.get("http://localhost:7187/").ok guards are **obsolete** since Phase 0 "use the live fixtures instead. tests/unit/test_no_runtime_service_skips.py enforces this.
- pytest-timeout hard-caps every test at 30 seconds. Stress and load tests that legitimately run longer must override with @pytest.mark.timeout(...).
- The CI workflow is **no longer manual-only** since Phase 0.4 "push and PR triggers the syntax / unit / integration workflows. Only the nightly.yml and security.yml are scheduled.

Test counts (machine-derived, Â§11.4.259)

Regenerated by [scripts/compute-badges.sh](#) â€” never hand-typed.
Corroborates the README badge row + the Contributing-section bullets.
Last regenerated: 2026-08-20T08:00:47Z.

Suite	Directory	Real count	Method
Python (whole tests/ tree)	tests/	5358	pytest -- collect-only -q
Python unit	tests/unit/	4406	pytest -- collect-only -q
Python integration	tests/integration/	288	pytest -- collect-only -q
Python e2e	tests/e2e/	30	pytest -- collect-only -q
Python contract	tests/contract/	12	pytest -- collect-only -q
Frontend (Vitest)	frontend/src/**/*.spec.ts	371	collected (vitest list â€”run)
HelixQA Challenges	challenges/scripts/*.sh	38	ls challenges/scripts/*.sh wc -l
Pre-build invariants	scripts/pre_build_verification.sh	30	max total of every [N/N] progress label

BOB-118 provenance note: the README badge row previously read python tests-585 passing / frontend tests-182 passing with no corroborating source anywhere in this document (a Â§11.4.6 bluff â€” the numbers were off by roughly 9x and could not be traced to any real invocation). This table is that corroborating source going forward; wording changed from â€œpassingâ€” to â€œcollectedâ€” because collection counts (this tableâ€™s method) prove existence, not that every test currently passes â€” a materially different, honestly-labelled claim.